

Instruction basics on the 2D fabric — the SAME opcode, Horizontal vs Vertical mode

One instruction per row. Colour tracks where data goes (numbers where the value matters). Flipping CSR 0x7c0 rotates each spatial op 90° — so a pixel is free to move anywhere on the grid. (vadd / vmv / vmerge are element-wise, so H = V.)

InputsHorizontal modeVertical mode (0x7c0=1)

vmv.v.x (broadcast)

vmv.v.x v8, a0

Copy one scalar into every lane.

a0 = 7

7	7	7	7
7	7	7	7
7	7	7	7
7	7	7	7

= same (H = V)

every lane = 7

vadd.vv

vadd.vv v8, vA, vB

Element-wise add. Position never moves.

1	2	3	4	+	4	3	2	1
5	6	7	8		8	7	6	5
2	4	6	8		8	6	4	2
1	3	5	7		7	5	3	1

5	5	5	5
13	13	13	13
10	10	10	10
8	8	8	8

= same (H = V)

A + B

vslideup.vi vd, vs, 1

csrwi 0x7c0,h/v vslideup.vi v8, vs, 1

Shift by one. Mode picks the axis.

shift RIGHT (within row)shift DOWN (between rows)

vrgather

vrgather.vx v8, vs, idx

Read any lane → any lane. Pixels are not local!

pixel at top-left

H: top-left → top-right V: top-right → bottom-right

vle8 / vse8

csrwi 0x7c0,1 vle8.v v8,(a0)

Vertical load/store transposes the image.

H: copy through V: transpose (rows ↔ cols)

vredsum.vs

vmv.v.i v0,0 vredsum.vs v8, vs, v0

Add up. Mode picks the axis the sum lands on.

1	2	3	4
1	1	1	1
2	0	2	0
4	3	2	1

10	.	.	.
4	.	.	.
4	.	.	.
10	.	.	.

8	6	8	6
.	.	.	.
.	.	.	.
.	.	.	.

H: row-sums in column 0 V: col-sums in row 0

vmerge.vvm + mask v0

vmerge.vvm v8, vB, vA, v0

Predicate per lane. Pick A where mask=1, else B.

mask gates the SAME lanes in H & V; only the data orientation under it changes

A where mask=1, else B